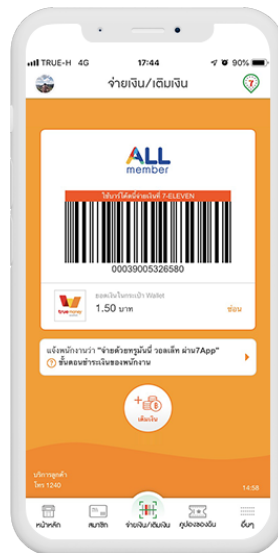


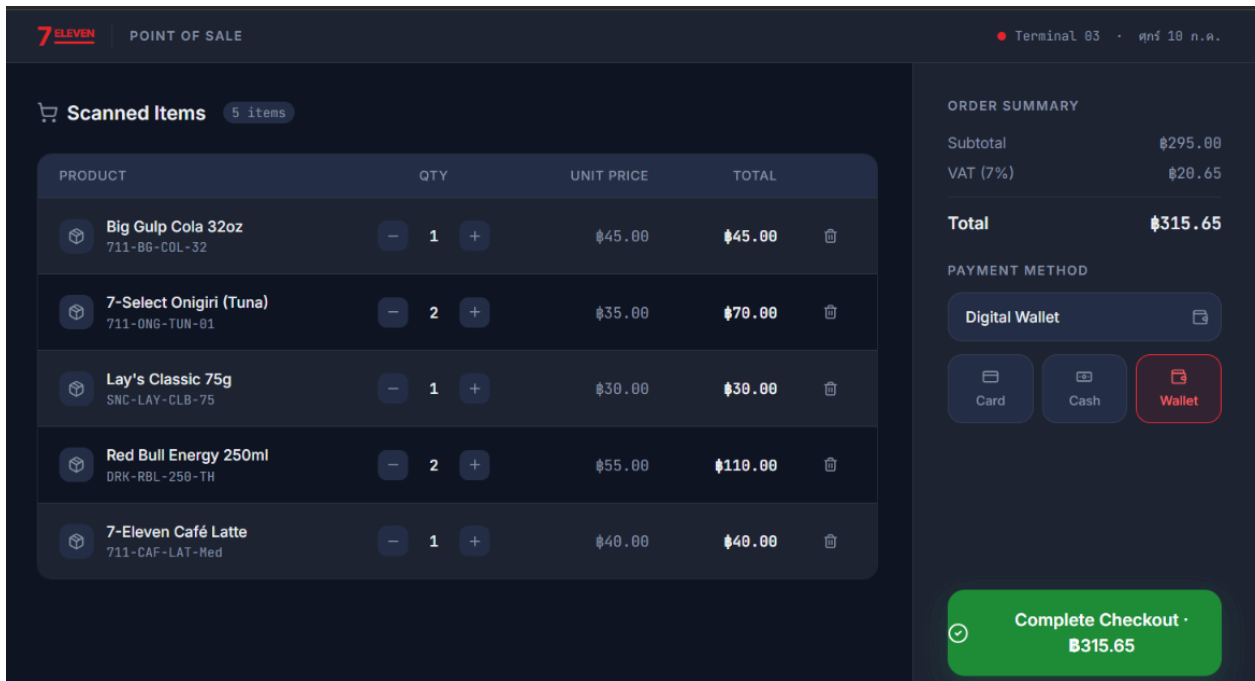
Screen Name: ALL Member Digital Wallet



- **Target User:** Customer
- **Primary Database View:** member_purchase_summary_view
- **How it Works:** When a customer opens their app, the interface queries the database using their unique member ID or phone number. Instead of running a complex, slow query to calculate their total history, the front-end pulls directly from member_purchase_summary_view. This view instantly gathers their phone number, active points balance, total visits, and lifetime spend.
- **Data Integrity Rules (Constraints):**
 - **chk_all_members_points_balance_non_negative:** The database prevents any transaction from deducting more points than a customer actually has. If an error occurs, the database blocks it, and the app UI safely displays a minimum of 0 points rather than a negative number.
- **Sample View Output (Using Seed Data):**

member_id	phone_number	points_balance	total_transactions	total_spent
1	0999999999	100	1	200.00

Screen Name: Cashier Point-of-Sale (POS) Checkout



Target User: Store Cashier (e.g., *Test Employee*)

Primary Database Tables: sales_transactions and transaction_items

How it Works: When a customer brings items to the register, the cashier scans them, creating list items in the UI. When the cashier clicks "Complete Checkout," the front-end executes two SQL INSERT statements to record the transaction details and the specific items sold.

Data Integrity Rules (Constraints):

- **chk_sales_transactions_payment_method:** The "Payment Method" drop-down in the UI is strictly limited to: *Cash, Credit Card or Wallet*. Any other input is blocked at the database level.
- **chk_transaction_items_quantity_positive:** The quantity of scanned items must be greater than zero, preventing checkout bugs like scanning "-1" of a product.

Behind-the-Scenes Automation (Database Triggers): The moment the transaction is recorded, the database instantly runs three critical operations on its own:

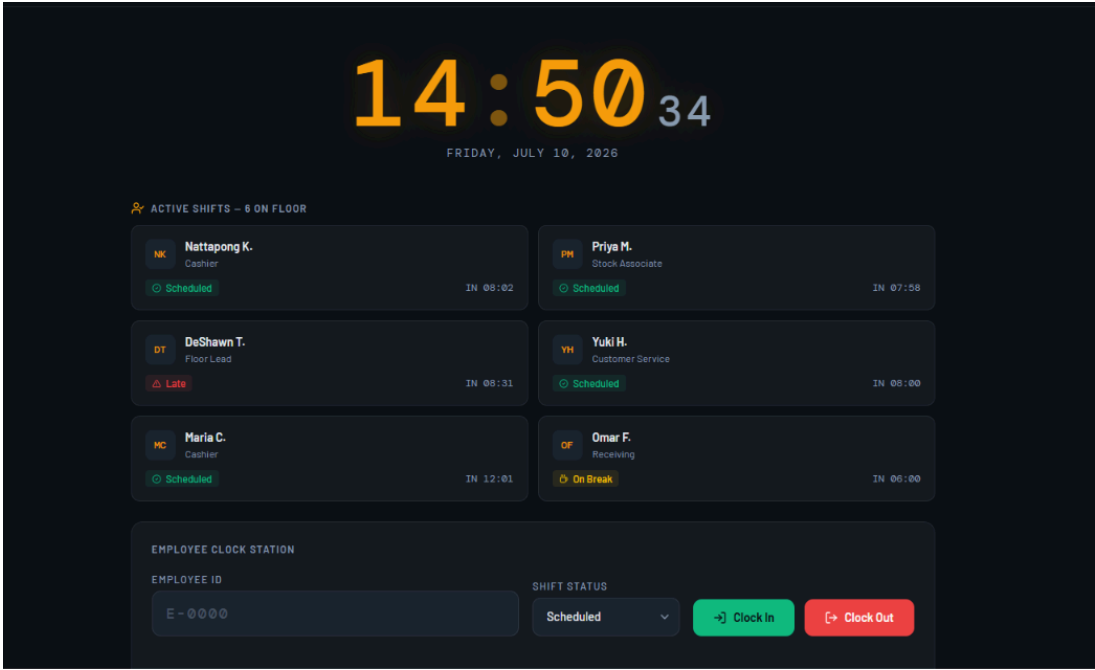
1. **trg_update_stock:** Automatically subtracts the sold items from store_inventory.
2. **trg_all_member_points:** If a member's phone number was entered, this trigger automatically divides the total purchase amount by 25, drops the decimal (using FLOOR), and adds those points to their balance.
3. **trg_stock_reorder:** If the item's stock drops below 10, the database immediately places a wholesale order to the supplier with a status of 'ORDER SENT'.

Sample Database Output (Using Seed Data):

transacti on_id	store _id	membe r_id	employe e_id	total_am ount	transaction _date	payment_m ethod
1	1	1	1	200.00	2026-07-10 (Current)	Cash

transaction_id	product_id	expiration_date	quantity	unit_price
1	1	2026-12-31	2	29.50

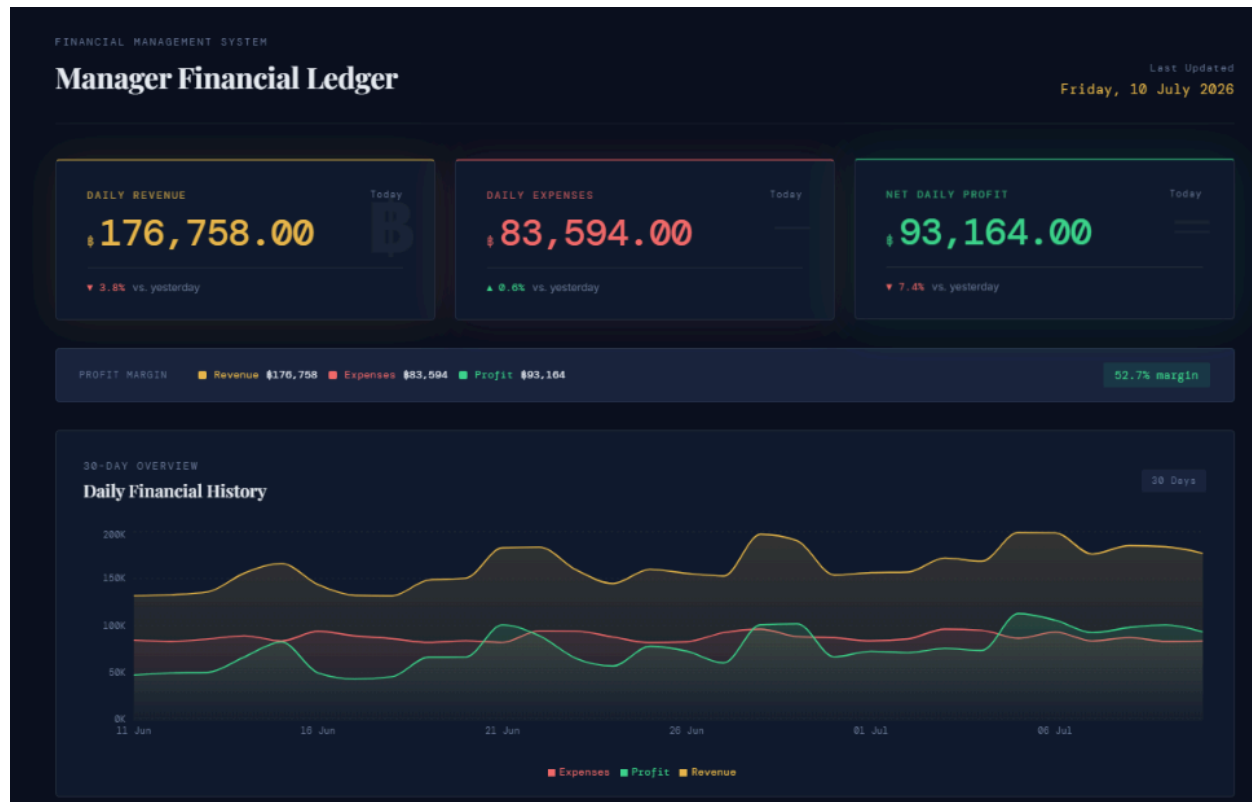
Screen Name: Employee Shift Punch Kiosk



- **Target User:** Store Employees and Branch Manager
- **Primary Database Objects:** worker_shifts (Table) and active_staff_view (View)
- **How it Works:** Employees use this back-room tablet to track their working hours. When an employee logs in, the screen displays the active_staff_view, instantly showing the manager who is currently on the floor (where clock_in is recorded but clock_out is NULL).
- **Data Integrity Rules (Constraints):**
 - **chk_worker_shifts_status:** The system strictly limits a shift's status to specific states: *Scheduled*, *Completed*, *Absent*, *Late*, or *Cancelled*. This prevents managers from typing in invalid or custom status codes.
 - **chk_worker_shifts_clock_out_after_clock_in:** The database provides foolproof time-theft and error protection. If a user accidentally attempts to submit a "Clock Out" time that occurred *before* their "Clock In" time, the PostgreSQL database outright rejects the insert.
- **Sample View Output (Using Seed Data via active_staff_view):**

store_id	employee_id	employee_name	role_type	clock_in
1	1	Test Employee	Cashier	2026-07-10 08:00:00+07

Screen Name: Manager Financial Ledger



- **Target User:** Store Manager / Franchise Owner
- **Primary Database View:** `manager_daily_net_profit`
- **How it Works:** This dashboard compiles three distinct database views to give managers a real-time, high-level summary of store performance. Instead of making the frontend calculate complex math, it directly queries `manager_daily_net_profit`.
 - In the background, `manager_daily_financials` aggregates daily store transactions.
 - Meanwhile, `manager_daily_expenses` sums up total store operational costs (such as utilities, logistics, or waste).
 - Finally, `manager_daily_net_profit` automatically joins these two views together on their respective dates to compute the net balance.
- **Data Integrity Rules (Constraints):**
 - **`chk_store_expenses_amount_non_negative`:** Prevents human error by ensuring any logged utility or operational expense amount is positive.
 - **`chk_sales_transactions_total_amount_non_negative`:** Guarantees that sales totals cannot accidentally be processed as negative values.
- **Sample View Output (Using Seed Data via `manager_daily_net_profit`):**

day_net_profit	net_profit_date
-300.00	2026-07-10 (Current Date)

Screen Name: Corporate Security & Audit Log

7-Eleven

Corporate Security

Overview

Audit Trail

Flagged Events

Live Monitor

Access Control

Settings

7-Eleven Corporate Audit Portal

TH-CORP / Bangkok Headquarters — System Access Level 4

LIVE

10 July 2026

Total Logs Today

1,284

+12% vs yesterday

Flagged Changes

37

+3 vs yesterday

Approved Edits

918

+8% vs yesterday

Pending Review

329

-4 vs yesterday

System Audit Trail

8 RECORDS

Search logs, products, users...

ALL

Flagged

Approved

Pending

Export

LOG ID	PRODUCT / TRANSACTION ID	CHANGE TYPE	OLD VALUE	NEW VALUE	CHANGED BY	TIMESTAMP	STATUS
AUD-2024-7881	SKU: TH-00432 Crystal Drinking Water 600ml	PRICE MOD	\$7.00	\$8.00 ↗	P. Pattaraporn Saengsuk Category Manager	2024-07-10 08:14:15	Flagged
AUD-2024-7882	T80-TH-99041 7-Cafe Latte (Hot)	EXPENSE MOD	\$45.00	\$39.00 ↘	N. Natthawut Jiraporn Store Supervisor	2024-07-10 09:02:17	Approved
AUD-2024-7883	SKU: TH-00089 7-Select Ready Meal Rice Box	PRICE MOD	\$35.00	\$42.00 ↗	S. Siriporn Charoenwong Regional Director	2024-07-10 10:11:44	Approved
AUD-2024-7884	T80-TH-99042 Singha Beer 330ml	EXPENSE MOD	\$55.00	\$48.00 ↘	K. Kittipong Ratchadamri Finance Officer	2024-07-10 11:20:05	Flagged
AUD-2024-7885	SKU: TH-01204 Oishi Green Tea 500ml	PRICE MOD	\$20.00	\$22.00 ↗	W. Wanida Thonglor Category Manager	2024-07-10 12:04:19	Approved
AUD-2024-7886	T80-TH-99088 Fried Chicken 2 Pieces	EXPENSE MOD	\$60.00	\$75.00 ↗	S. Supachal Vannavong Operations Manager	2024-07-10 13:55:00	Pending
AUD-2024-7887	SKU: TH-00301 Foremost UHT Milk 250ml	PRICE MOD	\$14.00	\$16.00 ↗	P. Pattaraporn Saengsuk Category Manager	2024-07-10 15:00:17	Approved
T80-TH-99181					Monsirol Proulx	2024-07-10	

- **Target User:** Regional Corporate Auditor / Franchise Owner
- **Primary Database Tables:** price_history_audit, expenses_audit, and transaction_audit
- **How it Works:** This secure interface serves as a read-only logging view for corporate security personnel. It monitors any unauthorized modifications or internal tampering. The frontend never has to calculate or manually push data to these tables; the logging happens entirely at the database layer.
- **Behind-the-Scenes Automation (Database Triggers):**
Whenever an internal record is updated or deleted, PostgreSQL silently intercepts the action and executes the corresponding auditing logic:
 - **trg_price_audit:** Automatically tracks if anyone updates a product's price, firing fn_price_audit() to capture the old_price, new_price, the database username (current_user), and the exact microsecond timestamp.
 - **trg_deleted_product_audit:** If a product row is purged from the catalog, this trigger immediately backs up the deleted record id into the archive table.

- **trg_expenses_audit / trg_transaction_audit**: Captures any updates made to financial expense entries or sales invoice values respectively.
- **Sample Database Audit Outputs (From Audit Schema Data):**

price_history_audit Table Entry:

audit_id	product_id	old_price	new_price	changed_by	change_time	change_date
1	1	25.00	29.50	postgres_admin	2026-07-10 15:13:48+07	2026-07-10

deleted_products_audit Table Entry:

audit_id	old_product_id	changed_by	change_time	change_date
1	99	cashier_user01	2026-07-10 11:24:02+07	2026-07-10